Plan of the chapter



1. Introduction
2. SQL data types
3. Common SQL Commands
4. SQL operators
5. Database and relational database design
6. Database normalization

SQL Arithmetic Operators

- In SQL, arithmetic operators are used to perform mathematical operations on data. To be more specific, they're used with numerical data stored in database tables.
- Arithmetic operators can be used in the SELECT clause as well as in the WHERE clause in a SQL SELECT statement.

- All arithmetic operators are used on numerical operands for performing:

- Addition
- Subtraction
- Multiplication
- Division
- Modulus

SQL Arithmetic Operators: Addition operator

- The SQL addition operator performs the mathematical addition operation on numerical data within columns in a table. The syntax is as follows:

```
SELECT column_name1 + column_name2 FROM table_name;
```

- **Example:** This is an employee table from a company database.
  You have to know the total salaries of all employees with the basic salary and the allowance added to it then you can use the addition operator.

employee_ID	employee_name	salary	allowance
1	Alex	25000	1000
2	John	55000	1000
3	James	52000	1000
4	Sam	30000	1000

- **The SQL syntax:** `SELECT salary + allowance FROM employee;`

SQL Arithmetic Operators: Addition operator

- The SQL addition operator performs the mathematical addition operation on numerical data within columns in a table. The syntax is as follows:

```
SELECT column_name1 + column_name2 FROM table_name;
```

- **Example 2:** This is the addition operator in the WHERE clause using the data in the following employee table.
  You have to retrieve the salaries of employees whose total salary is 25000.

employee_ID	employee_name	salary	allowance
1	Alex	24000	1000
2	John	55000	1000
3	James	52000	1000
4	Sam	24000	1000

➢ **The SQL syntax:**

```
SELECT* FROM employee WHERE salary+allowance=25000;
```

SQL Arithmetic Operators: Subtraction operator

- The SQL subtraction operator performs mathematical subtraction on numerical data within columns in a database table. The syntax is as follows:

```
SELECT column_name1 - column_name2 FROM table_name;
```

- **Example 1:** This is the employee table with a "tax" column and several instances of new data. You have to retrieve the salaries of employees after deducting tax.

employee_ID	employee_name	salary	allowance	tax
1	Alex	24000	1000	1000
2	John	55000	1000	2000
3	James	52000	1000	2000
4	Sam	24000	1000	1000

  - **The SQL syntax:** `SELECT salary - tax FROM employee;`

SQL Arithmetic Operators: Subtraction operator

- The SQL subtraction operator performs mathematical subtraction on numerical data within columns in a database table. The syntax is as follows:

```
SELECT column_name1 - column_name2 FROM table_name;
```

- **Example 2:** Consider the same employee table and data. You have to find out who earns a salary of 50000 after the tax deduction.

employee_ID	employee_name	salary	allowance	tax
1	Alex	24000	1000	1000
2	John	55000	1000	2000
3	James	52000	1000	2000
4	Sam	24000	1000	1000

- **The SQL syntax:**

```
SELECT * FROM employee WHERE salary - tax = 50000;
```

SQL Arithmetic Operators: Multiplication operator

- The SQL multiplication operator performs the mathematical multiplication operation on the numerical data typed columns in a database table.

Example 1: Consider the same employee table and data.

Let's say in the employee table, you want to generate the tax amounts for each employee if these amounts are doubled.

employee_ID	employee_name	salary	allowance	tax
1	Alex	24000	1000	1000
2	John	55000	1000	2000
3	James	52000	1000	2000
4	Sam	24000	1000	1000

- **The SQL syntax :**

```
SELECT tax * 2 FROM employee;
```

SQL Arithmetic Operators: Multiplication operator

- The SQL multiplication operator performs the mathematical multiplication operation on the numerical data typed columns in a database table.

Example 2: You want to know who must pay an amount of tax equal to 4000, after doubling the current tax value.

employee_ID	employee_name	salary	allowance	tax
1	Alex	24000	1000	1000
2	John	55000	1000	2000
3	James	52000	1000	2000
4	Sam	24000	1000	1000

- **The SQL syntax:**

```
SELECT * FROM employee WHERE tax * 2 = 4000;
```

SQL Arithmetic Operators: Division operator

- The division operator divides the numerical values of one column by the numerical values of another column. The syntax is as follows:

```
SELECT column_name1 / column_name2 FROM table_name;
```

Example 1: You have to find out the allowance percentage each employee receives, by using the salary and the allowance amount.

employee_ID	employee_name	salary	allowance	tax
1	Alex	24000	1000	1000
2	John	55000	1000	2000
3	James	52000	1000	2000
4	Sam	24000	1000	1000

- **The SQL syntax:**

```
SELECT allowance / salary * 100 FROM employee;
```

SQL Arithmetic Operators: Division operator

- The division operator divides the numerical values of one column by the numerical values of another column. The syntax is as follows:

```
SELECT column_name1 / column_name2 FROM table_name;
```

Example 2: Let's try to find out which of the employees get an allowance of at least 5%.

employee_ID	employee_name	salary	allowance	tax
1	Alex	24000	1000	1000
2	John	55000	1000	2000
3	James	52000	1000	2000
4	Sam	24000	1000	1000

➢ **Syntax:**

```
SELECT* FROM employee WHERE allowance / salary * 100>=5;
```

SQL Arithmetic Operators: Modulus operator

- The modulus operator (%) behaves as it's expected in SQL by giving the remainder when the numerical values of one column is divided by the numerical values of another column. The syntax is as follows:

```
SELECT column_name1 % column_name2 FROM table_name;
```

Example 1: You want to find out if the number of hours worked by each employee is an even number.

employee_ID	employee_name	salary	hours	allowance	tax
1	alex	24000	10	1000	1000
2	John	55000	11	3000	2000
3	James	52000	7	3000	2000
4	Sam	24000	11	1000	1000

- **The SQL syntax :**

```
SELECT hours % 2 FROM employee;
```

SQL Arithmetic Operators: Modulus operator

- The modulus operator (%) behaves as it's expected in SQL by giving the remainder when the numerical values of one column is divided by the numerical values of another column. The syntax is as follows:

```
SELECT column_name1 % column_name2 FROM table_name;
```

Example 2: You want to filter out only the employees who worked for an even number of hours:

employee_ID	employee_name	salary	hours	allowance	tax
1	alex	24000	10	1000	1000
2	John	55000	11	3000	2000
3	James	52000	7	3000	2000
4	Sam	24000	11	1000	1000

The SQL syntax:

```
SELECT * FROM employee WHERE hours % 2 = 0;
```

SQL Comparison Operators

- Comparison operators are useful when you want to write conditions in the WHERE clause of a SELECT statement to filter out records from a table.
- These comparison operators are:
  - `=` Checks for equality
  - `<>` or `!=` Checks for not inequality
  - `>` Check if something is greater than
  - `>=` Check if something is greater than or equal
  - `<` Check if something is less than
  - `<=` Check if something is less than or equal

SQL Comparison Operators: Equality operator

Example 1: You want to retrieve the data for the employee whose ID value is 1!

employee_ID	employee_name	salary	hours	allowance	tax
1	alex	24000	10	1000	1000
2	John	55000	11	3000	2000
3	James	52000	7	3000	2000
4	Sam	24000	11	1000	1000

- **The SQL syntax:**

```
SELECT * FROM employee WHERE employee_id = 1;
```

SQL Comparison Operators: Equality operator

Example 2: You need to retrieve the data for the employee whose name is James. You can use the equal operator in the WHERE clause condition.

employee_ID	employee_name	salary	hours	allowance	tax
1	alex	24000	10	1000	1000
2	John	55000	11	3000	2000
3	James	52000	7	3000	2000
4	Sam	24000	11	1000	1000

- **The SQL syntax:**

```
SELECT * FROM employee
WHERE employee_name = 'James';
```

SQL Comparison Operators: Inequality operator

Example 1: You have to determine which employee receives a salary that does not equate to 24000.

employee_ID	employee_name	salary	hours	allowance	tax
1	alex	24000	10	1000	1000
2	John	55000	11	3000	2000
3	James	52000	7	3000	2000
4	Sam	24000	11	1000	1000

The SQL syntax:

```
SELECT * FROM employee WHERE salary <> 24000;
```

SQL Comparison Operators: Greater than operator

Example 1: You have to find out which employees are earning a salary of over 50000.

employee_ID	employee_name	salary	hours	allowance	tax
1	alex	24000	10	1000	1000
2	John	55000	11	3000	2000
3	James	52000	7	3000	2000
4	Sam	24000	11	1000	1000

The SQL syntax:

```
SELECT * FROM employee WHERE salary > 50000;
```

SQL Comparison Operators: Greater than or equal operator

Example 1: You want to see who pays a tax amount of 2000 or more.

employee_ID	employee_name	salary	hours	allowance	tax
1	alex	24000	10	1000	1000
2	John	55000	11	3000	2000
3	James	52000	7	3000	2000
4	Sam	24000	11	1000	1000

The SQL syntax :

```
SELECT * FROM employee WHERE tax >= 2000;
```

SQL Comparison Operators: Less than operator

Example 1: You have to determine which employees get an allowance below 2500.

employee_ID	employee_name	salary	hours	allowance	tax
1	alex	24000	10	1000	1000
2	John	55000	11	3000	2000
3	James	52000	7	3000	2000
4	Sam	24000	11	1000	1000

The SQL syntax:

```
SELECT * FROM employee WHERE  allowance  < 2500;
```

SQL Comparison Operators: Less than or equal operator

Example 1: You have to determine which employees have worked for less than or equal to 10 hours.

employee_ID	employee_name	salary	hours	allowance	tax
1	alex	24000	10	1000	1000
2	John	55000	11	3000	2000
3	James	52000	7	3000	2000
4	Sam	24000	11	1000	1000

The SQL syntax:

```
SELECT * FROM employee WHERE  hours  <= 10;
```

Plan of the chapter



1. Introduction
2. SQL data types
3. Common SQL Commands
4. SQL operators
5. Database and relational database design
6. Database normalization

What is the relational model?

- The relational model is built around three main concepts which are:
  - **Data,**
  - **Relationships,**
  - **Constraints.**
- It describes a database as “a collection of inter-related relations (or tables)”.
- It is still a dominant model used for data storage and retrieval.
- It is a way of organizing or storing data in a database.
- SQL is the language that’s used to retrieve data from a relational database.

Fundamental concepts of the relational model

- **Relation:** It represents a file that stores data. It's also known as a **table**.
- Within a table there are **rows** and **columns**.
- Each row represents a group of related data values. A row, or record, is also known as a tuple. It consists of a set of attributes.
- Columns in a table are also known as fields or attributes. These columns define or describe a row.

	Column 1	Column 2	Column 3
Row 1			
Row 1			
Row 1			

Fundamental concepts of the relational model

❖ **Column:** A table stores pieces of data or facts as columns (attribute).

- When determining the columns for your table, think about the pieces of data that need to be stored within that table.
- Each column is a generic representation of the piece of data that needs to be stored.
- Each table cell that becomes a part of a row will have a specific instance of a piece of data.

ID	First Name	Last Name
1	John	Smith
2	Mish	Azerrad
3	Peter	Klien

Fundamental concepts of the relational model

- **Domain:** The domain is a set of acceptable values that a column is allowed to contain.
  - The domain depends on the data type of the column whether it is numeric, or text based.
  - The domain of ID has a set of acceptable and possible values that are numeric such as 1, 2 and 3.
  - The domain of First Name has a set of acceptable and possible values that are text based, which is people's first names.
  - In the ID column, it's not possible to store values such as "Jane" or "0001". Similarly, the First Name column can't accept any numeric pieces of data.

Fundamental concepts of the relational model

❖ **Record or tuple:** It is a row within a table.

- If a table has columns for ID, First Name and Last Name, then one record or tuple would have one person's ID, first name and last name.
- Another record would have another person's full personal information.

❖ **Key:** Each row or tuple has one or more attributes, known as a relation key, that can uniquely identify a specific row.

➢ This is also known as the **primary key**.

Fundamental concepts of the relational model

❖ **Degree:** It is the number of columns or attributes within a relation.

- A student table that stores the student's name, address, phone number and email address would have a degree of **4**.

❖ **Cardinality:** It refers to how many records there are within a particular table in a database.

- If you have 100 students in your student table, with all their information organized into individual rows, then that table has a cardinality of 100.

What are constraints?

- In the relational model, every relation needs to meet three conditions.
- These three conditions must be met for a relation to be valid.
- They are called relational integrity constraints and they are:
  - **Key constraints**
  - **Domain constraints**
  - **Referential integrity constraints**

What are constraints?

- **Key constraints:** It revolves around the key attribute(s).
- In the relational model, a key attribute is an identifier that can be used to refer to a record.
- A key attribute must also be unique for each record.
  - For example, you can use the Student ID in the student table as the key.
    - There can't be two students with the same Student ID.
    - It would be invalid and cause an issue when it comes to accessing or retrieving the data.
- A key attribute cannot have NULL values.

What are constraints?

- **Domain constraints:** There are all about the requirement of inserting values that have a valid data type.
- There are a variety of data types that can be included within a table, namely numeric, text and data.
- If an attempt is made to store an incorrect data typed value to an attribute, it's declared a violation of domain constraints.
- If an attribute requires a numeric value to be entered, and the value you are attempting to enter uses letters instead of numbers, then it would be invalid.

What are constraints?

- **Referential integrity constraints:** A database has multiple tables that refer to one another. Referential integrity constraints are based on the concept of **foreign keys**.
- A foreign key is a key attribute present in a table, which is also a primary key of another table to which it needs to be linked.
  - The foreign key references the other table to which it's related.
  - For example, the order ID is present in the Order_Item table as a foreign key, which is also the primary key of the order table.
  - So, the order table and the Order_Item table are related to each other because the Order_Item table references the order table via the order ID attribute.

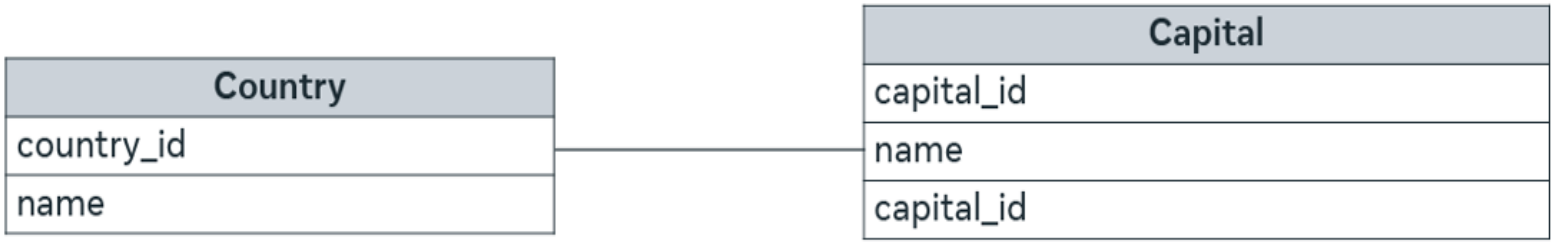
Types of relationships

- In the relational model, there are three types of relationships that can exist between tables:
  - **One-to-one**
  - **One-to-many**
  - **Many-to-many**

Types of relationships

❖ **One-to-one:** Let's take the example of two tables: Table A and Table B.

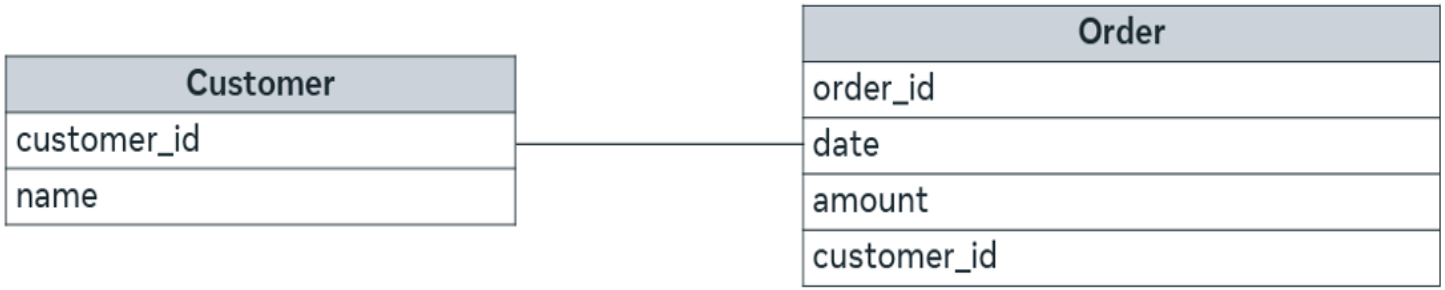
- A one-to-one (1:1) relationship means that each record in Table A relates to one, and only one, record in Table B. Likewise, each record in Table B relates to one, and only one, record in Table A.



- *Every country has one, and only one, capital.*
- *Every capital belongs to one and only one country.*

Types of relationships

- **One-to-many:** If there are two tables, Table A and Table B, a one-to-many (1:N) relationship means a record in Table A can relate to zero, one, or many records in Table B.
- Many records in Table B can relate to one record in Table A.



- *Each customer can place many orders.*
- *Many records in the order table can relate to only one record in the customer table.*

Types of relationships

- **Many-to-many:** If there are two tables, Table A and Table B, a many-to-many (N:N) relationship means many records in Table A can relate to many records in Table B. And many records in Table B can relate to many records in Table A.
- Many-to-many relationships are not kept in a data model. They are broken down into two one-to-many relationships by introducing a junction or middle table.

Customer
customer_id
name
product_id

Product
product_id
description
price
customer_id

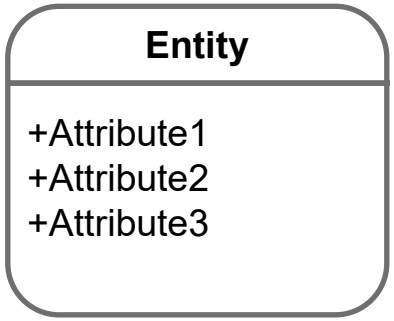
Entity relationship diagrams (ERD)

- The relational database design is very well connected with the entity relationship modelling process including entities, attributes and relationship identification and definition.
- The entity-relationship diagram is commonly used to represent and document the entity relationship models.
- The use of entity relationship diagrams helps to provide the big picture of your database.
- It ensures the data requirements and operations are well defined and documented in your project.

Entity relationship diagrams (ERD)

Entity representation:

- In the ERD, a box with two compartments is used to represent the entity and its related attributes.
- The top compartment represents the entity name, and the bottom compartment includes the related attributes.

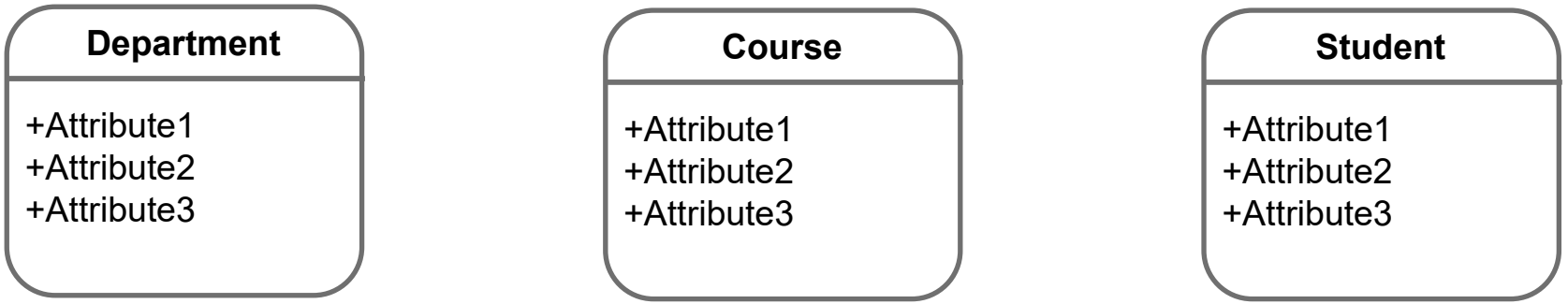


Entity relationship diagrams (ERD)

❖ **Entity representation:**

- Example, a college enrolment system contains a database with information about the students, and the courses available in each department.

➢ You have three entities represented in three separate boxes: the student entity, the course entity, and the department entity.



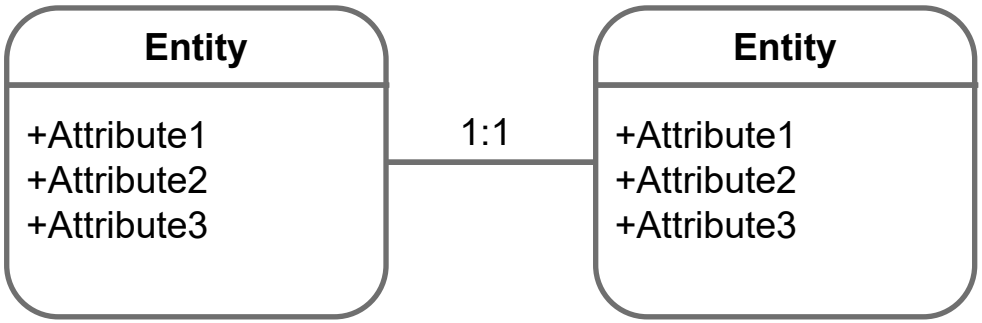
Entity relationship diagrams (ERD)

❖ **Relationship representation**

- The ERD uses different styles of lines to define the distinct types of relationships between entities.
- The line style depends on the cardinality of the relationship, which refers to the number of elements in a set of data as clarified in the following three cases.

❖ **1:1 (one-to-one):** The ER-D uses a straight-line representation for a one-to-one cardinality relationship.

- Example: Each passenger on a train should have only one ticket.

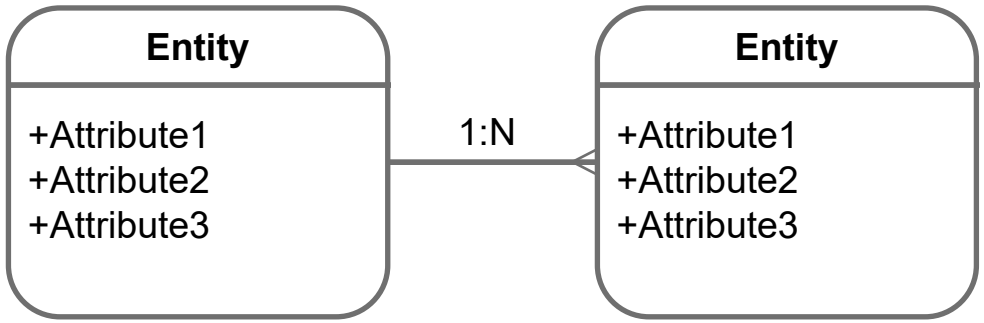


Entity relationship diagrams (ERD)

❖ Relationship representation:

1:N (one-to-many): The ERD is a straight line with a crow's foot notation on one side only to represent a one-to-many cardinality relationship.

- Example: One parent can have many children.

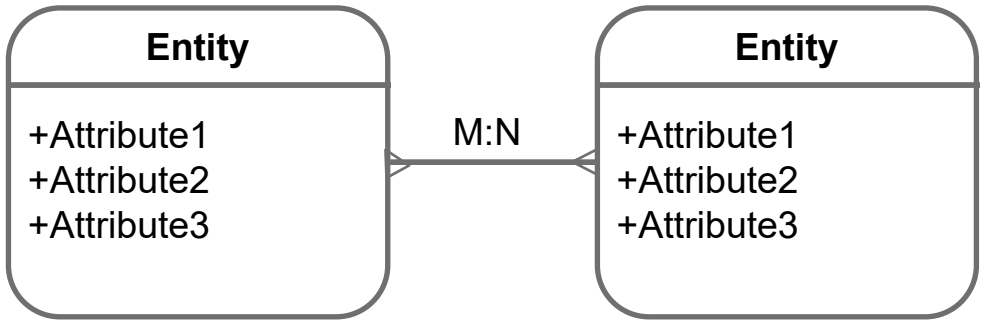


Entity relationship diagrams (ERD)

❖ Relationship representation:

M:N (many-to-many): The ER-D is a straight line with crow's foot notations on both sides of entities to represent a many-to-many cardinality relationship.

Example: Many players play many games.



Entity relationship diagrams (ERD)

❖ **Example:**

How would you depict the relationship between the student, course, and department entities introduced in the college enrolment system example?

Remember that many students may enrol in one course, and one department may offer many courses.

Entity relationship diagrams (ERD)

❖ **Attributes representation:**

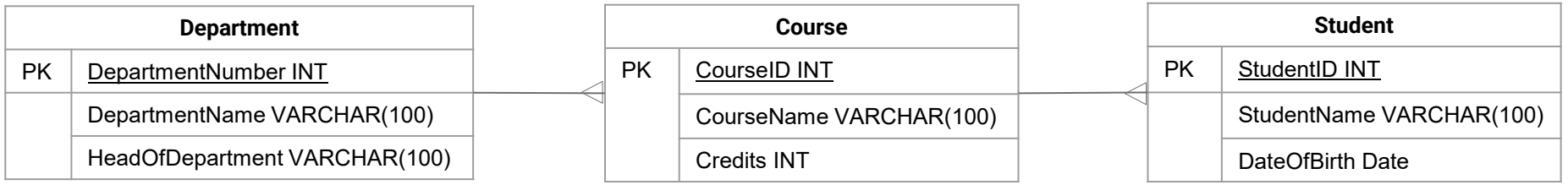
- Each entity has a set of attributes that hold relevant information about it.
- Each attribute must be defined with a data type.

- In the college enrolment example, list the following attributes followed by relevant data types:
  - The department attributes: department number, department name and head of department.
  - The course attributes: course ID, course name, and course credits.
  - The student attributes: student ID, name, and date of birth.

➢ These entities can be listed as three separate tables.

Entity relationship diagrams (ERD)

❖ Attributes representation:



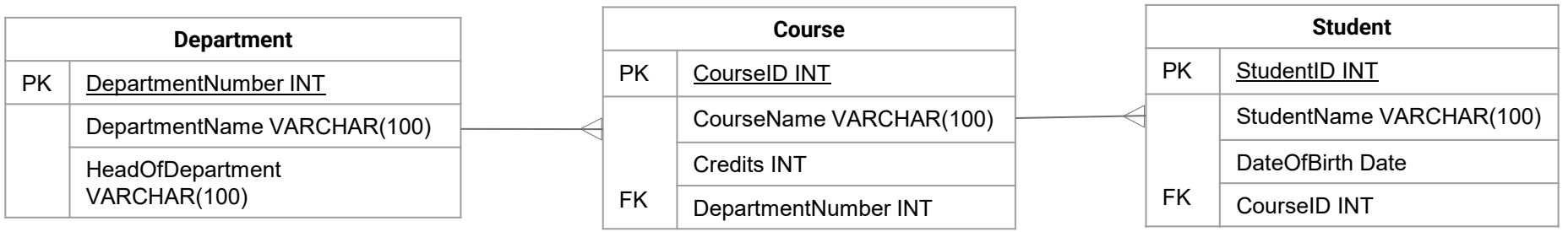
Entity relationship diagrams (ERD)

❖ Attributes representation:

- You need to add the department number to the course table as a **foreign key** in order to link the courses with the departments.
- You need to add the course ID to the student table as a `foreign` **key** in order to link the students with the courses.
- The final college enrolment entity relationship diagram is then three separate tables connected by the relevant keys.

Entity relationship diagrams (ERD)

❖ Attributes representation:



Building a database schema

- Building a database schema is the first step in database design.
- A database schema is like a blueprint of how data in a database will look and be stored.
- A schema consists of what's known as **schema objects** like tables, columns and relationships, at a minimum. Data types, views, stored procedures, primary keys and foreign keys are also schema objects.
- Basically, a database schema consists of:
  - all the important data pertaining to a given scenario and their relationships,
  - unique keys for all entries and database objects,
  - and a name and data type for each column in a table.

Building a database schema

❖ **Scenario:**

❑ To build a database schema for a restaurant booking system, you must consider that:

- Customers make reservations for tables and those tables have orders associated with them.
- An order will have associated menu items that belong to a menu.
- The orders are served by a waiter.

Building a database schema

❖ **The logical database schema:**

❑ Draw an Entity Relationship Diagram (ERD).

Building a database schema

❖ **The logical database schema:**

